

JavaScript APIs & Data Fetching Lecture Notes

1. What is an API?

Meaning: API stands for Application Programming Interface. It is a set of rules that allows one piece of software application to talk to another. In web development, we usually mean "Web APIs"—servers that provide data to our frontend applications when we ask for it.

Real-Life Example: Think of an API like a waiter in a restaurant. You (the client/frontend) look at the menu and give your order to the waiter (the API). The waiter takes your order to the kitchen (the server/database), gets your food, and brings it back to your table.

2. What is JSON?

Meaning: JSON stands for JavaScript Object Notation. It is a lightweight format for storing and transporting data. When a server sends data to your browser via an API, it almost always sends it in JSON format.

JSON vs. JavaScript Object

While they look very similar, there is a strict difference:

- JS Object: Used inside your JavaScript code. Keys do not need quotes.
- JSON: Is purely a STRING (text). It is strictly formatted. ALL keys and string values MUST be wrapped in double quotes ("").

Parsing and Stringifying

To use JSON data in JS, you have to convert it.

- `JSON.stringify()`: Converts a JS Object into a JSON string (used when sending data to a server).
- `JSON.parse()`: Converts a JSON string back into a usable JS Object (used when receiving data).

Code:

```
const myObj = { name: "Umar", age: 25 };

// Convert Object to JSON String
const jsonString = JSON.stringify(myObj);
console.log(jsonString); // '{"name":"Umar","age":25}'
```

```
// Convert JSON String back to Object
const parsedObj = JSON.parse(jsonString);
console.log(parsedObj.name); // Umar
```

3. XMLHttpRequest (The Old Way)

Meaning: Before modern methods existed, developers used XMLHttpRequest (XHR) to fetch data from servers without reloading the page. It is clunky and heavily relies on callbacks.

How to use it (open, send, onload, onerror)

Code:

```
const xhr = new XMLHttpRequest();

// 1. Configure the request: method (GET) and the URL
xhr.open('GET',
'https://jsonplaceholder.typicode.com/users/1');

// 2. What happens when the data loads successfully
xhr.onload = function() {
  if (xhr.status >= 200 && xhr.status < 300) {
    // Data comes back as a JSON string, so we parse it
    const data = JSON.parse(xhr.responseText);
    console.log("XHR Success:", data.name);
  } else {
    console.log("Server Error");
  }
};

// 3. What happens if there is a network error
xhr.onerror = function() {
  console.log("Network Error");
};

// 4. Send the request to the server
xhr.send();
```

4. The Fetch API

Meaning: `fetch()` is the modern, built-in JavaScript method for making network requests. It is much cleaner than XHR because it uses Promises (and works perfectly with `async/await`).

Code:

```
// Using fetch with async/await
async function getUser() {
  try {
```

```

        // 1. Fetch the data (returns a promise)
        const response = await
fetch('https://jsonplaceholder.typicode.com/users/1');

        // 2. Check if the response is OK (status 200-299)
        if (!response.ok) throw new Error('Network response
was not ok');

        // 3. Parse the JSON body of the response
        const data = await response.json();

        console.log("Fetch Success:", data.name);
    } catch (error) {
        console.error("Fetch Error:", error);
    }
}
getUser();

```

5. Axios

Meaning: Axios is a very popular third-party JavaScript library for making HTTP requests. Developers love it because it automatically converts JSON data for you and has better error handling out of the box compared to `fetch()`. (Note: You must install/import Axios to use it).

Code:

```

// Assuming Axios is imported via script tag or npm
async function getUserWithAxios() {
    try {
        // Axios automatically parses the JSON!
        const response = await
axios.get('https://jsonplaceholder.typicode.com/users/1');

        // The data is found inside the .data property
        console.log("Axios Success:", response.data.name);
    } catch (error) {
        // Error handling is easier with Axios
        console.error("Axios Error:", error.message);
    }
}
getUserWithAxios();

```

6. Real-World Example: Fetching Weather Data

In this example, we will use a hypothetical Weather API. Often, to get the weather for a city, you must first do a "Geocoding" request to turn a City Name into Latitude and Longitude coordinates. Then, you use those coordinates to fetch the actual weather.

```
Code:
const API_KEY = 'your_api_key_here';
const city = 'London';

async function getWeather() {
  try {
    // Step 1: Get Latitude and Longitude for the city
    const geoResponse = await
fetch(`https://api.openweathermap.org/geo/1.0/direct?q=${city}
&limit=1&appid=${API_KEY}`);
    const geoData = await geoResponse.json();

    if (geoData.length === 0) {
      throw new Error("City not found");
    }

    const lat = geoData[0].lat;
    const lon = geoData[0].lon;
    console.log(`Coordinates for ${city}: Lat ${lat}, Lon
${lon}`);

    // Step 2: Use the Lat and Lon to get the current
weather
    const weatherResponse = await
fetch(`https://api.openweathermap.org/data/2.5/weather?lat=${l
at}&lon=${lon}&units=metric&appid=${API_KEY}`);
    const weatherData = await weatherResponse.json();

    // Extract and log the final result
    const temperature = weatherData.main.temp;
    const description =
weatherData.weather[0].description;

    console.log(`Current weather in ${city}:
${temperature}°C, ${description}.`);

  } catch (error) {
    console.error("Failed to fetch weather:", error);
  }
}
```

```
getWeather();
```